Hybrid Movie Recommender

Cold-start handling, explained recommendations and a user-controlled discovery dial

Sahil Lenka (2427010391) · **Kanishk Shukla** (2427010432)

Recommender Systems mini project · Manipal University Jaipur
Live app: sahil-u07-hybrid-movie-recommender-app-dqy8sx.streamlit.app
Source: github.com/Sahil-u07/hybrid-movie-recommender

1. Summary

We built a movie recommender that works for a user with no history, explains every pick in one line, and lets the user set how adventurous the list is. It runs as a Streamlit web app on MovieLens data and is deployed publicly.

Six signals are combined into one score with weights that depend on how much the user has interacted. On a held-out time split the tuned hybrid beats a popularity baseline by 47% NDCG@10 for users with history and by 27% in a simulated cold start, both statistically significant. It matches collaborative filtering on accuracy rather than beating it; what it adds over that baseline is the cold-start path, the explanations and the diversity control.

2. Problem

Three failures motivated the design:

- **Cold start.** A new user has no ratings, so collaborative filtering has nothing to work with and most systems fall back to one generic popular list.
- **No reasons.** Users get a list of titles with no indication of why, so they cannot tell a good recommendation from a lucky one and stop trusting the system.
- **No control.** Ranking purely by predicted score returns ten near-identical films, and the user has no way to ask for something different tonight.

Our target: usable picks within a minute of arriving, each with a reason the user can check, and a control for how familiar or surprising the list should be.

3. Data

MovieLens `ml-latest-small` (GroupLens Research, 2016 release): 9,125 movies, 100,004 ratings, 671 users and 1,296 tag applications. Each film carries genre labels, and `links.csv` carries IMDb and TMDB identifiers, which we use to fetch poster artwork.

The project specification describes the 2018 release. That file was not reachable from our build environment, so we used the 2016 release. The schema is identical and the code runs on either without modification.

4. Method

4.1 The six signals

Every candidate film is scored on six signals. Each signal is min-max scaled to [0, 1] inside the candidate set, so signals on different scales can be combined.

Signal	How it is computed
Preference	Cosine similarity between the film's genre vector and the genres chosen in the quiz
Content	TF-IDF over genre tokens and MovieLens tags, cosine similarity to liked films; disliked films subtract
Collaborative	Item-item adjusted cosine on user-mean-centred ratings, shrunk by n/(n+10) so pairs with few common raters count less
Context	The selected mood mapped to a set of genres
Quality	Bayesian average rating blended with log rating count
Discovery	Genre unfamiliarity multiplied by how well the film still matches taste and quality

A liked film contributes +1.0 to the taste vector and a disliked film -0.7, the event weights from the specification. A film is a candidate if it has at least 5 ratings and the user has not already seen, liked or disliked it. Avoided genres are removed before scoring, as a hard filter rather than a penalty.

4.2 Profile stages and weights

The weights depend on how much the system knows about the user: **cold** (no interactions), **warming** (1 to 19) and **established** (20 or more). Collaborative filtering carries no weight in the cold stage because it has no signal there, and grows as interactions arrive.

Signal	Cold	Warming spec	Warming tuned	Established spec	Established tuned
Preference	0.450	0.300	**0.130**	0.200	**0.098**
Content	0.150	0.250	**0.057**	0.250	**0.065**
Collaborative	0.000	0.100	**0.424**	0.250	**0.448**
Context	0.150	0.150	0.150	0.150	0.150
Quality	0.150	0.100	**0.139**	0.050	**0.139**
Discovery	0.100	0.100	0.100	0.100	0.100

> **On tuning.** The weights in the specification were untuned starting values, and with them the hybrid scored *below* plain popularity. We tuned the warming and established columns by random search on a validation slice cut out of the training data, never the test split. Quality is capped at 0.15 because the specification calls it a tiebreaker; an uncapped search pushed it to 0.40 and started returning generic hits to a user who had picked five science fiction films. Every signal keeps a floor of 0.05. The cold stage has no interaction data to tune against, so it keeps the specification values.

4.3 Diversity: MMR re-ranking

Ranking by score alone returns near-duplicates, because the highest-scoring films cluster together. We re-rank with Maximal Marginal Relevance: each next film is chosen by its score minus its largest similarity to the films already in the row.

```
pick = argmax [ lambda * score(film) - (1 - lambda) * max similarity(film, already chosen) ]
```

The discovery dial sets lambda and the discovery weight together: Comfort zone 0.85 / 0.05, A little different 0.70 / 0.10, Surprise me 0.55 / 0.20. No single genre may take more than half of a row.

4.4 Explanations

Each pick stores the weighted contribution of every signal. The two largest contributions fill a reason template, and a signal contributing less than 10% of the score is never cited. Because the text is generated from the stored breakdown rather than written separately, a reason can never name a signal that did not actually affect the ranking. The user can open the breakdown and see the same numbers the sentence came from.

5. System

```
app.py           Streamlit interface: quiz, billboard, rows, breakdown, profile pages
recommender.py   Model: candidate set, six signals, stage weights, MMR, explanations
eval.py          Offline evaluation, baselines and paired significance tests
tune.py          Weight search on a validation split of the training data
tests/           17 tests, including a headless run of the interface
data/            MovieLens CSVs
results/         Evaluation output used in this report
```

The public entry point is one function:

```
recommend(profile, context, k=10) -> list[dict]
# profile: {"genres": [...], "liked": [movie_ids], "disliked": [movie_ids]}
# context: {"mood": "funny" | None, "discovery": "comfort" | "different" | "surprise"}
# result:  movie_id, title, score, breakdown, explanation, reasons, genres, avg_rating,
n_ratings
```

Poster artwork is fetched from the film's English Wikipedia article, which needs no API key. The app asks for exact article titles, most specific first (`The Matrix (1999 film)` , then `The Matrix (film)` , then `The Matrix`) and follows redirects. Wikipedia's own search was not reliable for this: asked for "Pulp Fiction 1994 film" it returned the soundtrack album, and Terminator 2 returned a list of video games. A whole row of artwork resolves in one request and is cached, misses included. If a TMDB API key is supplied the app uses TMDB instead, matching on the TMDB id rather than any title lookup.

6. The application

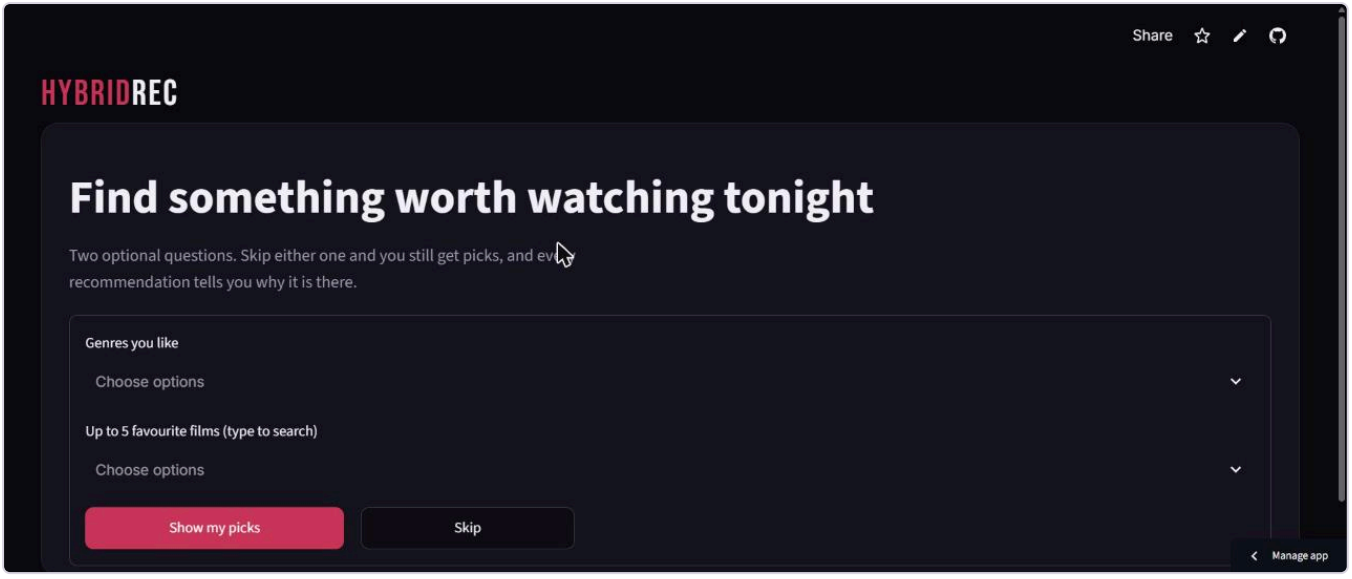


Figure 1. The opening quiz. Both questions are optional and the whole step can be skipped, in which case the recommender starts from the quality signal alone.

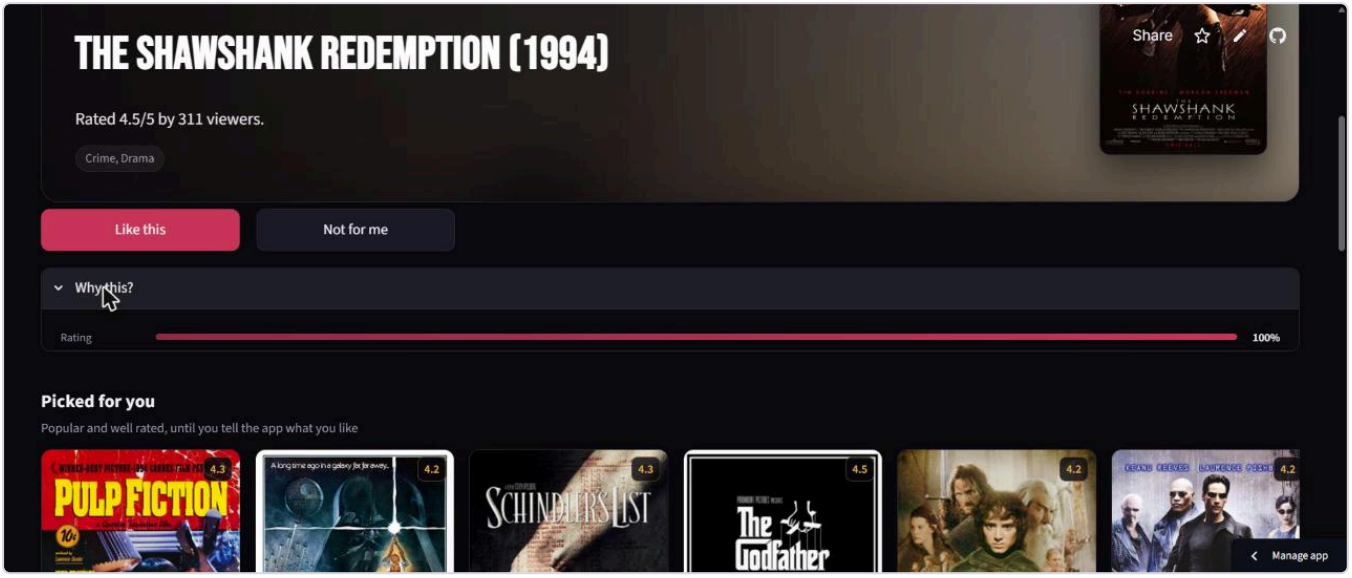


Figure 2. The top pick, with its reason and the score breakdown opened. The bar shows that in the cold stage this pick rests entirely on the quality signal, which is what the weights say should happen.

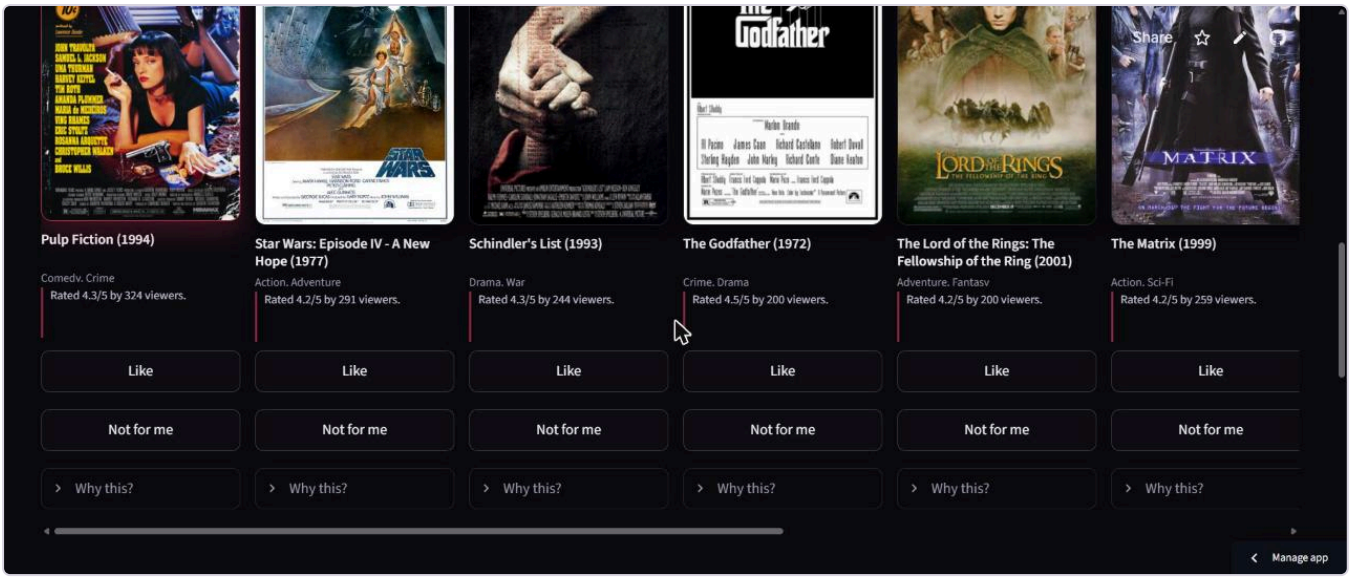


Figure 3. A row of recommendations. Each card carries its reason, and Like or Not for me re-ranks the whole page immediately.

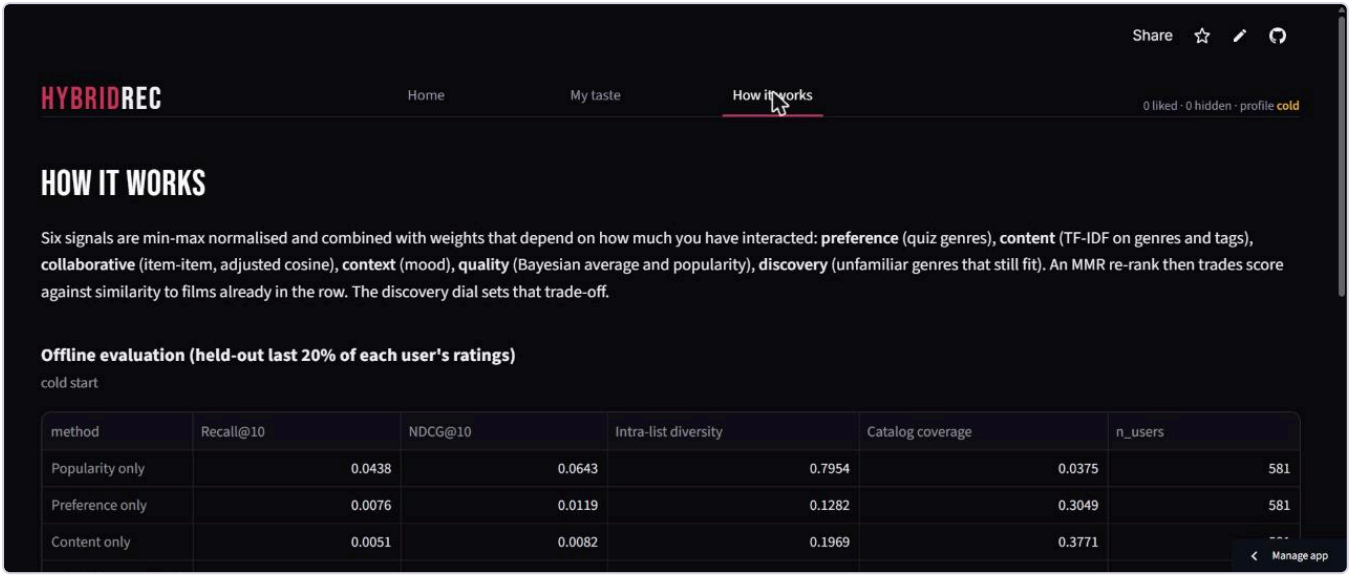


Figure 4. The app exposes its own offline evaluation, so the numbers in this report can be checked in the running system.

7. Evaluation

7.1 Setup

- **Split.** Per user by time: the first 80% of each user's ratings train the models, the last 20% are held out as the test set.
- **Relevance.** A held-out film rated 4 or higher counts as a hit.
- **No leakage.** Item statistics, content similarity and the collaborative model are built from training ratings only. Films a user rated during training are hidden from every method.
- **Cold start.** Simulated by showing the model only each user's five best-rated training films and the top three genres of those films.

7.2 Metrics

- **Recall@10.** The share of the user's held-out liked films that appear in the top ten.

- **NDCG@10.** Rewards liked films appearing near the top, divided by the best possible ordering, so it runs from 0 to 1.
- **Intra-list diversity.** Mean genre distance between the ten films in a list.
- **Catalog coverage.** The share of eligible films the method ever recommends.

7.3 Results: users with history (585 users with 20 or more ratings)

Method	Recall@10	NDCG@10	Intra-list diversity	Catalog coverage
Popularity only	0.0455	0.0649	0.7956	4%
Preference only	0.0100	0.0137	0.1137	21%
Content only	0.0088	0.0122	0.1945	30%
Collaborative only	0.0631	0.0945	0.7643	17%
Hybrid (spec weights)	0.0311	0.0522	0.3974	24%
Hybrid (tuned)	**0.0657**	**0.0956**	**0.7243**	**13%**
Hybrid (tuned) + MMR	**0.0569**	**0.0873**	**0.8266**	**13%**

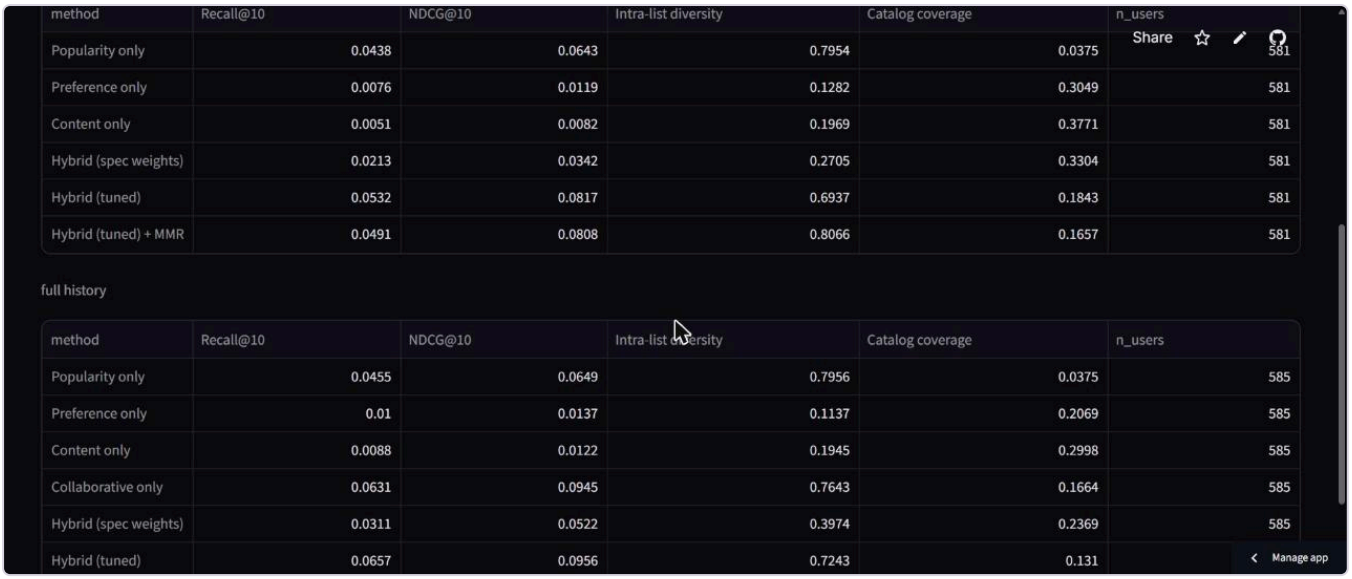
7.4 Results: cold start (581 users, five favourites and their top three genres)

Method	Recall@10	NDCG@10	Intra-list diversity	Catalog coverage
Popularity only	0.0438	0.0643	0.7954	4%
Preference only	0.0076	0.0119	0.1282	30%
Content only	0.0051	0.0082	0.1969	38%
Hybrid (spec weights)	0.0213	0.0342	0.2705	33%
Hybrid (tuned)	**0.0532**	**0.0817**	**0.6937**	**18%**
Hybrid (tuned) + MMR	**0.0491**	**0.0808**	**0.8066**	**17%**

7.5 Significance

Paired t-tests across users on NDCG@10. The first four compare on full history, the last two on cold start.

- Hybrid (tuned) vs Popularity only: NDCG diff +0.0307 (+47%), p=0.0000
- Hybrid (tuned) vs Collaborative only: NDCG diff +0.0012 (+1%), p=0.6290
- Hybrid (tuned) vs Content only: NDCG diff +0.0835 (+686%), p=0.0000
- Hybrid (tuned) + MMR vs Hybrid (tuned): NDCG diff -0.0083 (-9%), p=0.0014
- Hybrid (tuned) vs Popularity only: NDCG diff +0.0174 (+27%), p=0.0022
- Hybrid (tuned) vs Content only: NDCG diff +0.0736 (+899%), p=0.0000



method	Recall@10	NDCG@10	Intra-list diversity	Catalog coverage	n_users
Popularity only	0.0438	0.0643	0.7954	0.0375	581
Preference only	0.0076	0.0119	0.1282	0.3049	581
Content only	0.0051	0.0082	0.1969	0.3771	581
Hybrid (spec weights)	0.0213	0.0342	0.2705	0.3304	581
Hybrid (tuned)	0.0532	0.0817	0.6937	0.1843	581
Hybrid (tuned) + MMR	0.0491	0.0808	0.8066	0.1657	581

full history

method	Recall@10	NDCG@10	Intra-list diversity	Catalog coverage	n_users
Popularity only	0.0455	0.0649	0.7956	0.0375	585
Preference only	0.01	0.0137	0.1137	0.2069	585
Content only	0.0088	0.0122	0.1945	0.2998	585
Collaborative only	0.0631	0.0945	0.7643	0.1664	585
Hybrid (spec weights)	0.0311	0.0522	0.3974	0.2369	585
Hybrid (tuned)	0.0657	0.0956	0.7243	0.131	585

Figure 5. The same evaluation tables as rendered inside the running app.

8. Discussion

What the numbers support. The tuned hybrid beats the popularity baseline by a wide and significant margin in both settings, and beats content-only by a very large margin. The cold-start result matters most for the original goal: with only five films and three genres, the hybrid is 27% better than recommending the same popular list to everyone.

What they do not support. The hybrid does not beat item-item collaborative filtering on accuracy. The difference is about 1% and the paired test returns $p = 0.63$, so the honest conclusion is that they are equal on this data. The hybrid's advantage over that baseline is that it works for a user with no ratings at all, it explains its picks, and it exposes a diversity control. Pure collaborative filtering does none of those.

The accuracy and variety trade. MMR costs about 9% of NDCG@10 and returns about 0.10 of genre diversity. That is a genuine trade rather than a free gain, which is exactly why it is a user-facing dial instead of a fixed constant.

9. Limitations

- Catalog coverage is 13%, below the 20% target in our specification. The untuned weights reach 24% coverage but at much lower accuracy, so coverage and accuracy are in tension here.
- The dataset is small and older than the one the specification names, and the weights were tuned on a single split of 671 users.
- Content similarity is weak on this data. Most films carry only genre tokens and few have tags, which shows in the content-only baseline.
- MovieLens small has no runtime or language fields, so those filters were not built.
- The system has not been tested with real users, so the online metrics in the specification (pick rate, completion rate, negative feedback rate) are not measured.
- Poster artwork from Wikipedia is non-free material used there under fair use. It is acceptable for a course demonstration; a public deployment should use TMDB, which licenses artwork for this purpose.

10. Future work

- Move to the full stack in the specification: Next.js, FastAPI and Postgres with pgvector.

- Train the collaborative component on MovieLens 25M and add embeddings from film overviews.
- Add the franchise cap, so a row cannot fill with sequels of one film.
- Group recommendation for more than one viewer, and free-text requests.
- Measure online metrics with real users rather than a held-out split.

11. Running the project

```
python -m venv .venv
.venv\Scripts\activate          # macOS or Linux: source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py             # the web app

python -m pytest -q tests        # 17 tests
python eval.py                   # reproduces every number in section 7
python tune.py                   # repeats the weight search
```

The MovieLens data ships with the project, so nothing is downloaded on first run.

12. References

- F. Maxwell Harper and Joseph A. Konstan. "The MovieLens Datasets: History and Context." *ACM Transactions on Interactive Intelligent Systems* 5(4), 2015.
- J. Carbonell and J. Goldstein. "The use of MMR, diversity-based reranking for reordering documents and producing summaries." *SIGIR*, 1998.
- B. Sarwar, G. Karypis, J. Konstan and J. Riedl. "Item-based collaborative filtering recommendation algorithms." *WWW*, 2001.
- Poster artwork from English Wikipedia. Optional artwork and metadata from TMDB; this product uses the TMDB API but is not endorsed or certified by TMDB.